SIMATS ENGINEERING

Department of Information Technology

CSA3711 – Software Testing

CODE OF CONDUCT FOR ASSESSMENT SUBMISSION

Every student must read and follow this code of conduct.

1. The work submitted is my own individual work; this assignment is of an individual nature.
2. I have not copied from another student, from previous batches or from any other source without proper acknowledgement.
1. Any book, website, tool or software used for assistance has been acknowledged, and I understand the content I submit and can explain it in a viva.
2. I have not shared my solution with any other student and have not allowed anyone to copy it.
3. I have not fabricated or altered data, calculations or references. All calculations are shown and can be reproduced.
4. I will follow the submission format prescribed by the department: Times New Roman, 1.5 line spacing, answers written after each question.
5. I will submit the assignment on or before the due date; late submissions are subject to the rules of the department.
6. I accept that any violation of this code may lead to reduction or cancellation of marks and disciplinary action as per institutional rules.

Student Declaration

I have read the code of conduct and declare that this submission follows it.

Student Name	Rakcha A	**Register Number**	192311421
Department / Section	CSE	**Academic Year / Semester**	IV YEAR
Faculty Name	Kesavan R	**Nature of Assignment**	Individual
Duration / Total Marks	1 Week / 50	**Date of Submission**	

Rakcha A **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

Software Testing (CSA37)

ASSIGNMENT 2 – CO2

CO2: Test case design strategies, black-box testing, random testing, equivalence class partitioning, boundary value analysis, other black-box techniques, white-box testing, test adequacy criteria, coverage, control flow graphs, path testing.

Assignment Title: Design, Evaluate and Defend a Test Suite for a Courier Shipping-Charge Calculator (Total Marks – 50)

Duration: 1 week

Scenario — Courier Shipping-Charge Calculator (SwiftShip)

A courier company is launching SwiftShip, an online service that calculates the shipping charge for a parcel and tracks it until delivery. More than 40,000 parcels are booked every day. An undercharge causes revenue loss; an overcharge causes customer complaints and refunds.

You are the Test Designer. You must design test cases, evaluate their effectiveness and defend the final test suite that will be used before every release.

Shipping-charge rules (specification extract):

Rule	Specification
S1	Weight (kg, up to one decimal place) must be greater than 0 and not more than 50. Otherwise the booking is rejected as INVALID.
S2	Charge is calculated for every started kilogram: Zone A 40/kg, Zone B 60/kg, Zone C 90/kg. Any other zone is INVALID.
S3	Express delivery adds 50 % to the charge. Express is allowed only up to 30 kg; above 30 kg the request is NOT_ALLOWED.
S4	Customers who booked 10 or more parcels in the current month receive a 10 % discount on the final charge.

Shipping-charge module – Pseudocode under test:

```
     function shipCharge(weight, zone, express, parcels)
1    if weight <= 0 or weight > 50 then
2       return INVALID
3    rate = 40
4    if zone == "B" then
5       rate = 60
```

```
6       else if zone == "C" then
7           rate = 90
8       charge = 0
9       w = weight
10      while w > 0 do
11          charge = charge + rate
12          w = w - 1
13      if express then
14          if weight > 30 then
15              return NOT_ALLOWED
16          charge = charge * 1.5
17      if parcels >= 10 then
18          charge = charge * 0.9
19      return charge
```

Parcel tracking states:

BOOKED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED. A parcel can move from OUT_FOR_DELIVERY to FAILED_ATTEMPT, and from FAILED_ATTEMPT back to OUT_FOR_DELIVERY (maximum two retries) or to RETURNED. A booking can be CANCELLED only before PICKED_UP.

Design Challenge:

SwiftShip will add a fuel surcharge and weekend delivery, and the shipping-charge code will be modified. Re-evaluate your test suite and determine whether it should be retained, modified or redesigned.

Task 1 – Select Test Case Design Strategies 5 Marks

CO2 | Bloom's Level: K5 – Evaluate

SwiftShip has a calculation module (shipCharge) and a tracking module.

Which test design strategy will find the most important defects in each module with the least effort?

Your answer must include:

- The difference between black-box and white-box test design strategies
- The strategy you select for the shipping-charge rules, for the pseudocode and for the tracking states
- The information each strategy needs (specification, code, state model)
- The kinds of defects each strategy is likely and unlikely to reveal
- A justification for combining strategies rather than using only one

You are expected to select and justify strategies independently for this system.

Evidence Expected

A justified selection of test design strategies for each part of SwiftShip, with the expected strengths and weaknesses of each.

Black-box and white-box design compared

A **black-box** strategy derives tests from what the software is supposed to do (the rules S1–S4 or the state model) and ignores how the code is written. A **white-box** strategy derives tests from how the code is built (its statements, decisions and paths) and measures how much of that structure the tests have exercised. They answer different questions: black-box asks "does the product do what was promised?", white-box asks "has every part of what was written actually been run?".

Table 1: Black-box and white-box strategies applied to SwiftShip

Aspect	Black-box (specification-based)	White-box (code-based)
Test basis	Rules S1–S4, state model of parcel tracking	Pseudocode of shipCharge (lines 1–19), control flow graph
Main techniques	Equivalence classes, boundary values, decision table, state-transition	Statement, branch, condition coverage, basis-path testing, loop testing

Aspect	Black-box (specification-based)	White-box (code-based)
Typical defects found	Missing rules (invalid zone not rejected), wrong boundary (>= instead of >), wrong rate, illegal state changes	Unexecuted code, wrong logic inside a branch, loop errors, infeasible paths, hidden conditions
Defects it cannot find	Code that is not described in the specification; how much of the code was never run	Missing functions or rules (nothing to cover), wrong specification, omitted validations
Pre-requisite	A complete and unambiguous specification	Access to the source code and a coverage tool

Strategy selected for each part of SwiftShip

Table 2: Strategy selected for each part of SwiftShip and the information it needs

Part under test	Primary strategy	Information needed	Why it gives the best value for effort
Shipping-charge rules S1–S4	Black-box: equivalence classes + boundary values + decision table for S3/S4	The written rules (S1–S4)	Four short rules with numeric limits (0, 30, 50, 9/10) and one rule interaction; 7 + 13 + 8 tests cover all of it; most money defects sit at these edges or in missing rules
shipCharge pseudocode	White-box: control flow graph, basis paths (9) and branch/condition coverage	Source code or pseudocode	Shows which lines and decisions the black-box tests never reached and gives an objective stopping measure; only 9 basis tests
Tracking states	Black-box, model-based: state-transition testing (valid + selected invalid transitions)	State model and the rules for retries and cancellation	Behaviour depends on history, not on input ranges, so ECP/BVA do not apply; 8 valid and 48 invalid transitions give a finite, checkable test target

Likely and unlikely defects of each strategy

Table 3: Strengths and weaknesses of the strategies for SwiftShip

Strategy	Likely to reveal	Unlikely to reveal
Equivalence classes	Zone D accepted instead of INVALID; wrong rate for a zone; express or discount missing	Defects exactly on a boundary (the representative values sit inside the class)
Boundary values	Weight 30.0/30.1 and 50.0/50.1 mistakes; started-kilogram errors; parcels 9 versus 10	Wrong zone rate, because all boundary tests may use one zone
Decision table	Wrong combination of express, weight and discount	Defects in a single rule that does not combine with others
State-transition	Illegal transitions allowed; retry counter or cancellation errors	Calculation errors
Path / branch coverage	Unexecuted statements, wrong branch logic, loop errors	Missing validation (the invalid-zone omission), wrong boundary constants if the chosen data are not at the boundary

Why strategies must be combined

No single strategy covers both kinds of fault. In this project the seeded evidence is clear (Tasks 3, 9 and 10): the real defect in shipCharge, an unrecognised zone treated as zone A, is an **omission**, so only a black-box test (zone "D") can find it; the 9 basis-path tests cover 100% of branches yet miss it. Conversely, the basis-path tests alone kill only 9 of 12 injected faults and the black-box tests alone (ECP) only 8 of 12, while ECP + BVA + decision table kill all 12. The strategies are therefore complementary: black-box tests find what is wrong or missing against the specification, white-box measurement shows what remains unexercised, and state-transition tests cover the one module (tracking) that neither of them suits. Combining them costs only about 30 tests in total, which is small compared with the cost of one undercharged or overcharged parcel multiplied by 40,000 parcels a day.

Task 2 – Evaluate Random Testing 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

A team member suggests generating 1,000 random weights and zones instead of designing test cases.

Can random testing replace systematic test design for shipCharge?

Discuss:

- How random testing would be carried out for shipCharge
- The probability that random inputs hit important values such as 30.0, 30.1 and 50.0
- The problem of deciding the expected result (test oracle) for random inputs

Your judgement must be supported by reasoning, not opinion.

Evidence Expected

A reasoned evaluation of the usefulness and limits of random testing for shipCharge.

How random testing would be carried out

A generator draws each input from its allowed domain: weight from 0.0 to 60.0 in steps of 0.1 (601 values, so that invalid weights also occur), zone from a set of letters, express as true/false with equal probability, and parcels from 0 to 20. The program is run 1,000 times and every result is compared with an expected value. I assume a seeded generator so that a failing test can be repeated.

Probability of hitting the important values

Each draw hits one particular weight with probability 1/601. The chance that a value occurs at least once in 1,000 draws is 1 − (1 − p)^1000.

Table 4: Chance that 1,000 random tests include the important boundary inputs

Important input	Probability per draw	Probability in 1,000 draws	Calculation
Weight 50.0 (or 50.1)	1/601 = 0.17%	81.1%	1 − (600/601)^1000
Weight 30.0 with express = true	1/1,202 = 0.083%	56.5%	1 − (1,201/1,202)^1000
Weight 30.1 with express = true	1/1,202 = 0.083%	56.5%	1 − (1,201/1,202)^1000

Important input	Probability per draw	Probability in 1,000 draws	Calculation
Parcels = 9, parcels = 10	1/21 = 4.8% each	≈ 100%	1 − (20/21)^1000
All six of the above in one run of 1,000 tests	–	**about 21%** (simulation of 5,000 runs)	joint event

The weights are therefore hit by chance in only about half to four fifths of runs, and all the boundary conditions together in only about one run in five. By contrast, 13 designed boundary tests hit every one of them with certainty. If the zone is drawn only from A, B and C, the invalid-zone defect can never be found; if it is drawn from all letters, a random test reaches zone “D” easily, but then the oracle problem below decides whether the failure is recognised.

The oracle problem

A random input has no expected result attached. For 1,000 inputs the expected charge must be computed by a second program that implements S1–S4 independently (for example ceil(weight) × rate, × 1.5, × 0.9). If that program is written by the same developer or copies the logic of shipCharge, it repeats the same defect (the zone “D” charge of 40 per kg would be “expected”), so the 1,000 tests pass and the defect stays hidden. Checking 1,000 results by hand is impractical. Only property checks are cheap: the charge is never negative; express is exactly 1.5 times non-express; a discounted charge is exactly 90% of the undiscounted one; the charge never decreases when weight increases within a zone.

Judgement

Random testing **cannot replace** systematic design for shipCharge. It reaches the critical boundaries only by chance (about 21% for all of them), it needs an independent oracle that is costlier than designing 30 tests, and it has no coverage target to decide when to stop. It is still useful as a **complement**: a nightly run of random inputs with the property checks above can reveal crashes and unexpected values, and if the generator copies the real distribution of parcels it can estimate how often customers would meet a defect.

Task 3 – Design and Evaluate Equivalence Classes 6 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using only the specification S1–S4, apply Equivalence Class Partitioning.

What is the smallest set of test cases that represents every valid and invalid behaviour of the shipping-charge rules?

Discuss:

- Valid and invalid equivalence classes for weight, zone, express and parcels
- How you identified each class from the specification
- Test cases covering all valid classes with as few tests as possible
- Test cases covering each invalid class one at a time, and why
- Expected results calculated from the specification
- Which of your test cases would fail against the pseudocode, and what that reveals

Simply listing classes will not receive full credit. You are expected to evaluate the effectiveness of your classes.

Evidence Expected

A complete equivalence class table, derived test cases with expected results, and an evaluation of the defects they expose.

1. Equivalence classes and how they were identified

I read each input condition in S1–S4 and split every input domain into sets of values that the specification treats in the same way. A split is made wherever the specified behaviour changes, not wherever the numbers look different.

Table 5: Equivalence class table for the shipping-charge rules

Class	Input	Range / value	Valid?	Identified from
W1	Weight	$0 < w \leq 30$	Valid	S1 (valid range) and S3 (express allowed up to 30 kg)
W2	Weight	$30 < w \leq 50$	Valid	S1 valid; S3 changes the behaviour above 30 kg (express NOT_ALLOWED)
W3	Weight	$w \leq 0$	Invalid	S1: “greater than 0”, otherwise INVALID

Class	Input	Range / value	Valid?	Identified from
W4	Weight	w > 50	Invalid	S1: “not more than 50”, otherwise INVALID
W5	Weight	More than one decimal place (for example 10.55)	Invalid (spec gap)	S1 says “up to one decimal place” but gives no result for a violation
Z1, Z2, Z3	Zone	A, B, C	Valid	S2: three rates (40, 60, 90 per kg); each rate is a separate behaviour
Z4	Zone	Any other value (D, blank, lower case)	Invalid	S2: “any other zone is INVALID”
E1	Express	No	Valid	S3: no surcharge
E2	Express	Yes, weight ≤ 30	Valid	S3: +50%
E3	Express	Yes, weight > 30	Valid input, NOT_ALLOWED result	S3: express allowed only up to 30 kg
P1	Parcels	0 – 9	Valid	S4: no discount
P2	Parcels	10 or more	Valid	S4: 10% discount on final charge
P3	Parcels	Negative or not a whole number	Spec gap	S4 does not say; to be clarified

There are 9 valid classes (W1, W2, Z1–Z3, E1, E2, P1, P2) plus the derived NOT_ALLOWED behaviour E3, and 3 invalid classes with a stated result (W3, W4, Z4); W5 and P3 are specification gaps which I report instead of guessing an expected result.

2. Smallest set covering all valid classes

A test may cover one class of every input at once, so the valid classes can be covered with three tests; I add one more for the derived NOT_ALLOWED behaviour.

Table 6: Test cases covering the valid classes

Test	Input (weight, zone, express, parcels)	Classes covered	Expected result from the specification	Result on pseudocode
T1	4.5, A, No, 3	W1, Z1, E1, P1	5 started kg × 40 = **200**	Pass
T2	12.0, B, Yes, 10	W1, Z2, E2, P2	12 × 60 = 720; × 1.5 = 1,080; × 0.9 = **972**	Pass

Test	Input (weight, zone, express, parcels)	Classes covered	Expected result from the specification	Result on pseudocode
T3	40.0, C, No, 12	W2, Z3, E1, P2	40 × 90 = 3,600; × 0.9 = **3,240**	Pass
T4	35.0, A, Yes, 2	W2, E3	Express above 30 kg: **NOT_ALLOWED**	Pass

3. Invalid classes, one at a time

Each invalid test uses valid values for all other inputs, so that if it fails, the failure can only be caused by the invalid value being tested (the "one invalid at a time" rule). Combining two invalid values would let one mask the other.

Table 7: Test cases for the invalid classes

Test	Input	Class	Expected result	Result on pseudocode
I1	−2.0, A, No, 3	W3	INVALID	Pass (line 1)
I2	60.0, A, No, 3	W4	INVALID	Pass (line 1)
I3	5.0, **D**, No, 3	Z4	INVALID	**FAIL: returns 200**
I4	10.55, A, No, 3	W5	Not defined by S1–S4: raise a query	Returns 11 × 40 = 440 (no check)

The minimal ECP suite is therefore **7 tests** (T1–T4, I1–I3), plus I4 as a specification query. 7 tests is far fewer than the 96 combinations of classes (4 weight × 4 zone × 3 express × 2 parcels) that exhaustive combination would need.

4. Evaluation of the classes

- **Defect exposed:** I3 fails. The pseudocode (lines 3–7) sets rate = 40 and changes it only for "B" and "C"; there is no branch for an invalid zone, so zone D, a blank or "b" are charged at zone A's rate. This is a missing-validation defect: the customer is undercharged or a wrong booking is accepted, which is a revenue loss. Only an invalid-class test can reveal it.
- **Coverage obtained:** the 7 ECP tests alone execute 19 of 19 statements, 14 of 14 branch outcomes and 16 of 16 condition outcomes (Task 9), so a small black-box set already exercises every part of the code.
- **Limit of ECP found by experiment:** I injected 12 non-equivalent faults into the pseudocode (for example weight > 50 changed to ≥ 50). The ECP set killed 8 of 12. The

four survivors (≤ changed to <, > 50 to ≥ 50, > 30 to ≥ 30 and to > 31) are all boundary faults, because ECP representatives (4.5, 12.0, 40.0) lie inside the classes. This justifies adding boundary value analysis in Task 4.

- **Effectiveness of the partitioning:** the class boundaries were chosen where behaviour changes (30 and 50 kg, 10 parcels), so the same classes feed the decision table in Task 5 without change.

Task 4 – Design and Evaluate Boundary Value Tests 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Defects in charge calculations often occur at the edges of ranges.

Which boundaries in SwiftShip are most likely to hide defects, and which form of BVA is worth its cost?

Discuss:

- Boundary values for weight (0, 0.1, 30, 30.1, 50, 50.1), the started-kilogram rule and parcels (9, 10)
- Normal BVA, robust BVA and worst-case BVA, and the number of test cases each produces
- Test cases with expected results for the boundaries you select
- Which form of BVA you recommend for shipCharge, and why

You must justify the choice of BVA form against the number of test cases and the risk.

Evidence Expected

A justified boundary value test design with expected results and a defended choice of BVA form.

1. The boundaries that are most likely to hide defects

Table 8: Boundaries of SwiftShip and the defect each one is likely to hide

Boundary	Values to test	Likely defect (and cost)
Weight lower limit (S1)	0.0 (invalid), 0.1 (valid), 0.2	< written instead of ≤ at line 1: weight 0 accepted with charge 0 (revenue loss)
Started kilogram (S2)	1.0 → 1 kg; 1.1 → 2 kg; also 29.9, 30.0, 30.1	Rounding down instead of up: every parcel with a fraction is undercharged by one kg rate
Express limit (S3)	30.0 allowed; 30.1 NOT_ALLOWED; 29.9 allowed	> 30 written as ≥ 30: valid 30 kg express parcels refused
Weight upper limit (S1)	49.9, 50.0 valid; 50.1 INVALID	> 50 written as ≥ 50: the heaviest valid parcels rejected
Discount threshold (S4)	Parcels 9 (no discount); 10 (discount)	>= 10 written as > 10: the 10th parcel does not get the 10% discount, a complaint on every tenth booking

2. Forms of BVA and the number of tests they produce

Weight and parcels are the two numeric inputs (zone and express are categories and belong to ECP). Parcels has no upper limit in the specification; for counting I assume 0–99. With n = 2 variables:

Table 9: Number of test cases produced by each form of BVA

Form	Values per variable	Tests (n = 2)	Formula
Normal BVA	min, min+, nominal, max−, max	9	4n + 1
Robust BVA	Normal + min− and max+ (invalid sides)	13	6n + 1
Worst-case BVA	All combinations of the 5 normal values	25	5^n
Robust worst-case BVA	All combinations of the 7 robust values	49	7^n

3. Selected boundary tests and expected results

Unless stated otherwise the other inputs are zone A, express No, parcels 3 (so no surcharge and no discount); this isolates the boundary under test.

Table 10: Boundary value test cases with expected results

Test	Input (weight, zone, express, parcels)	Boundary	Expected result	Result on pseudocode
B1	0.0, A, No, 3	min−	INVALID	Pass
B2	0.1, A, No, 3	min	1 kg × 40 = **40**	Pass
B3	1.0, A, No, 3	kg step	1 kg × 40 = **40**	Pass
B4	1.1, A, No, 3	kg step	2 kg × 40 = **80**	Pass
B5	49.9, A, No, 3	max−	50 kg × 40 = **2,000**	Pass
B6	50.0, A, No, 3	max	50 kg × 40 = **2,000**	Pass
B7	50.1, A, No, 3	max+	INVALID	Pass
B8	29.9, A, Yes, 3	express limit −	30 × 40 = 1,200; × 1.5 = **1,800**	Pass
B9	30.0, A, Yes, 3	express limit	30 × 40 = 1,200; × 1.5 = **1,800**	Pass
B10	30.1, A, Yes, 3	express limit +	**NOT_ALLOWED**	Pass
B11	30.1, A, No, 3	no express above 30	31 × 40 = **1,240**	Pass
B12	5.0, A, No, 9	parcels 9	5 × 40 = **200** (no discount)	Pass
B13	5.0, A, No, 10	parcels 10	200 × 0.9 = **180**	Pass

The started-kilogram rule was verified by executing the loop on all 500 weights 0.1 to 50.0: every one gives ceil(weight) iterations, so the subtraction of 1 has no floating-point error at the one-decimal values the specification allows.

4. Recommended form of BVA

I recommend **robust BVA on weight (7 values), applied at each boundary that the specification defines, plus the two parcels values 9 and 10**: the 13 tests B1–B13. This is the same number as the formula for robust BVA (13), but the values are placed where the rules change (0, 30, 50, the kilogram step, 9/10) and not generated mechanically.

- **Why not normal BVA (9 tests):** it never tests the invalid sides 0.0 and 50.1, which is exactly where S1 can be broken.
- **Why not worst-case (25) or robust worst-case (49):** worst-case BVA is justified when variables interact. In shipCharge weight and parcels never appear in the same decision (weight in lines 1, 10 and 14, parcels in line 17), so combining their boundaries finds nothing new. The interaction that does exist, express × weight × parcels, is handled by the decision table (Task 5) with 8 tests, which is cheaper than 36 additional worst-case tests.
- **Risk and evidence:** the 13 BVA tests killed 10 of 12 injected faults, including all four that ECP missed. The two survivors are wrong zone rates (60 → 50, 90 → 80), because the BVA tests use zone A only; they are caught by the ECP tests T2 and T3, which is why the techniques are combined.

Task 5 – Apply Other Black-Box Techniques 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Express, weight and discount rules interact with each other.

Do ECP and BVA miss defects caused by combinations of rules?

Discuss:

- A cause–effect graph or decision table for rules S3 and S4
- Test cases derived from the decision table
- Combinations that ECP/BVA did not cover, and whether they matter

You are expected to compare the results with your ECP/BVA test set.

Evidence Expected

A decision table or cause–effect graph with derived tests and a comparison with ECP/BVA coverage.

1. Causes, effects and decision table

Causes: C1 = express requested, C2 = weight > 30, C3 = parcels ≥ 10. Effects: E1 = add 50%, E2 = NOT_ALLOWED, E3 = apply 10% discount. The cause–effect logic is: E2 = C1 AND C2; E1 = C1 AND NOT C2; E3 = C3 AND NOT E2 (a rejected request gets no discount). With three conditions there are 8 combinations.

Table 11: Decision table for rules S3 and S4

Rule	C1 Express	C2 Weight > 30	C3 Parcels ≥ 10	E1 Surcharge × 1.5	E3 Discount × 0.9	Result
R1	No	No	No	–	–	base
R2	No	No	Yes	–	Yes	base × 0.9
R3	No	Yes	No	–	–	base
R4	No	Yes	Yes	–	Yes	base × 0.9
R5	Yes	No	No	Yes	–	base × 1.5
R6	Yes	No	Yes	Yes	Yes	base × 1.5 × 0.9 = base × 1.35
R7	Yes	Yes	No	–	–	NOT_ALLOWED
R8	Yes	Yes	Yes	–	–	NOT_ALLOWED (discount not applied)

Here “base” = started kilograms × zone rate. R7 and R8 give the same result, but I keep both because the code checks the express limit after the loop and before the discount, and a defect that applied the discount first would show only in R8.

2. Test cases derived from the table

Table 12: Decision-table test cases

Test	Input (weight, zone, express, parcels)	Rule	Expected result	Result on pseudocode
DT1	10.0, A, No, 3	R1	10 × 40 = **400**	Pass
DT2	10.0, A, No, 10	R2	400 × 0.9 = **360**	Pass
DT3	40.0, B, No, 3	R3	40 × 60 = **2,400**	Pass
DT4	40.0, B, No, 10	R4	2,400 × 0.9 = **2,160**	Pass
DT5	10.0, C, Yes, 3	R5	10 × 90 = 900; × 1.5 = **1,350**	Pass
DT6	10.0, C, Yes, 10	R6	1,350 × 0.9 = **1,215**	Pass
DT7	40.0, B, Yes, 3	R7	**NOT_ALLOWED**	Pass
DT8	40.0, B, Yes, 10	R8	**NOT_ALLOWED**	Pass

3. Comparison with the ECP/BVA set

Table 13: Which decision-table rules the ECP and BVA tests cover

Rule	Covered by ECP (T1–T4)	Covered by BVA (B1–B13)	Covered by ECP + BVA
R1	T1	B2–B6, B12	Yes
R2	–	B13	Yes
R3	–	B11	Yes
R4	T3	–	Yes
R5	–	B8, B9	Yes
R6	T2	–	Yes
R7	T4	B10	Yes
R8	–	–	**No**

ECP alone covers only 4 of the 8 rules (R1, R4, R6, R7) because each ECP test combines several classes at once. BVA adds R2, R3 and R5 as a by-product of its boundary values, so

ECP + BVA together cover 7 of 8; only **R8** (express, over 30 kg, with the discount) is missed. The decision table therefore **does** find a combination that the other two techniques left out.

Does it matter? For the current code the missing combination is low risk: the discount is a multiplication and multiplication is commutative, so applying the surcharge and the discount in a different order gives the same charge; a defect there would be invisible whatever tests are used. R8 matters only if a change makes the discount apply to a NOT_ALLOWED request, for example after future code is added; it costs one test, so I keep it. Combinations with zone (4 zones × 8 rules = 32) are not needed because the zone only changes the rate and has no decision in common with S3 and S4.

Task 6 – Design and Evaluate State-Transition Tests 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

The tracking module behaves differently depending on the current state of a parcel.

Which invalid transitions could cost the company money or reputation if they were allowed?

Discuss:

- A state-transition diagram and table for parcel tracking
- Test cases for all valid transitions and selected invalid transitions
- Tests for the “maximum two retries” rule and the cancellation rule

You are expected to justify which invalid transitions you selected.

Evidence Expected

A state-transition model with justified valid and invalid transition tests.

1. State-transition model

I read the specification as follows. A booking starts in BOOKED. Cancellation is possible only from BOOKED (before PICKED_UP). A failed delivery attempt can be retried at most **twice**, so a parcel can have at most three delivery attempts (the first and two retries); after the third failure the only way out is RETURNED. This reading of “maximum two retries” is my assumption and should be confirmed with the business.

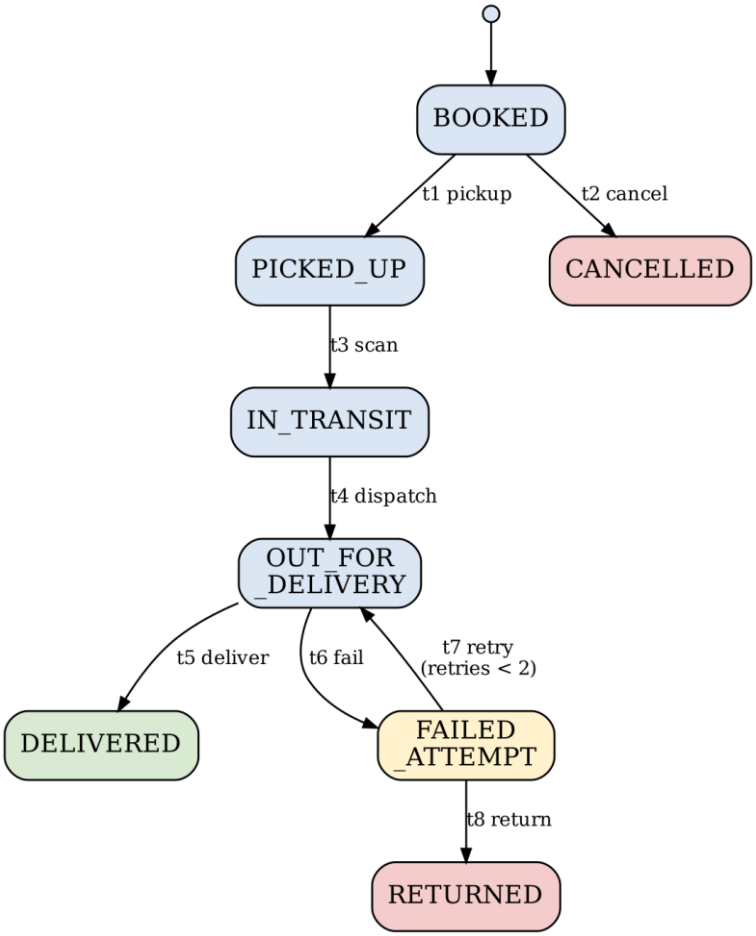


Figure 1: State-transition diagram of parcel tracking (t7 allowed only while retries < 2)

Table 14: State-transition table (all other event / state pairs are not allowed)

No.	Current state	Event	Next state	Condition
t1	BOOKED	pickup	PICKED_UP	–
t2	BOOKED	cancel	CANCELLED	only before PICKED_UP
t3	PICKED_UP	scan	IN_TRANSIT	–
t4	IN_TRANSIT	dispatch	OUT_FOR_DELIVERY	–
t5	OUT_FOR_DELIVERY	deliver	DELIVERED	–
t6	OUT_FOR_DELIVERY	fail	FAILED_ATTEMPT	–
t7	FAILED_ATTEMPT	retry	OUT_FOR_DELIVERY	retries so far < 2
t8	FAILED_ATTEMPT	return	RETURNED	–
–	DELIVERED, RETURNED, CANCELLED	any	none	final states: no outgoing transitions

There are 8 states and 8 valid transitions (t1–t8). Of the 8 × 7 = 56 ordered pairs of different states, 56 − 8 = 48 are invalid.

2. Tests for all valid transitions

Table 15: Valid-transition tests (the 8 transitions are covered by three sequences)

Test	Sequence of events	Transitions covered	Expected result
S1	BOOKED → pickup → scan → dispatch → deliver	t1, t3, t4, t5	Ends in DELIVERED; every step accepted
S2	BOOKED → cancel	t2	Ends in CANCELLED
S3	… OUT_FOR_DELIVERY → fail → retry (retries = 1) → fail → retry (retries = 2) → fail → return	t6, t7 twice, t8	Both retries accepted; ends in RETURNED

3. Selected invalid transitions and why

Forty-eight invalid transitions cannot all be tested at the same priority. I selected the ones that would cost money (refunds, free service, extra delivery cost) or reputation (a false "delivered" message), and left out harmless ones such as the backward step IN_TRANSIT → PICKED_UP.

Table 16: Selected invalid-transition tests

Test	Attempted transition	Why it is dangerous	Expected result
IT1	PICKED_UP → CANCELLED	Cancelling after pickup (cancellation rule): refund for a parcel the courier already holds	Rejected; state unchanged
IT2	IN_TRANSIT → CANCELLED	Same: refund after cost incurred	Rejected
IT3	OUT_FOR_DELIVERY → CANCELLED	Same, at the last mile	Rejected
IT4	BOOKED → DELIVERED	False "delivered" message; customer reputation and dispute	Rejected
IT5	IN_TRANSIT → DELIVERED	Skips the delivery attempt; proof of delivery missing	Rejected
IT6	FAILED_ATTEMPT → DELIVERED	Delivered without an attempt	Rejected
IT7	DELIVERED → RETURNED	Refund claim on a delivered parcel	Rejected

Test	Attempted transition	Why it is dangerous	Expected result
IT8	DELIVERED → OUT_FOR_DELIVERY	Re-delivery cost of a delivered parcel	Rejected
IT9	RETURNED → OUT_FOR_DELIVERY	Redelivery after return without a new booking	Rejected
IT10	CANCELLED → PICKED_UP	Pickup of a cancelled (possibly unpaid or refunded) booking	Rejected
IT11	FAILED_ATTEMPT → OUT_FOR_DELIVERY when retries = 2 (third retry)	Extra deliveries beyond the limit: cost and customer irritation	Rejected; only RETURNED allowed

4. Tests for the retry limit and the cancellation rule

- **Retry rule:** S3 shows that two retries are accepted (retries = 1 and 2); IT11 shows that the third is rejected. Boundary thinking applies here as well: the values 0, 1, 2 and 3 of the retry counter are the boundaries, and S3 + IT11 test 2 and 3 directly (1 is tested within S3).
- **Cancellation rule:** S2 (cancel in BOOKED, allowed) is the valid side; IT1–IT3 test PICKED_UP, IN_TRANSIT and OUT_FOR_DELIVERY, the nearest states after BOOKED and the later ones. Cancelling an already DELIVERED parcel is covered by the general rule that final states have no outgoing transitions (IT7, IT8 for DELIVERED).

The module therefore has **14 tests** (3 sequences + 11 invalid transitions): all 8 states and all 8 valid transitions (100%), and 11 of the 48 invalid transitions (23%), chosen by risk.

Task 7 – Construct the Control Flow Graph 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Using the pseudocode of shipCharge, construct the control flow graph.

Does the structure of the code match the structure of the specification?

Discuss:

- Nodes and edges, with line numbers mapped to nodes
- Decision nodes, the loop, and the compound condition at line 1
- Differences between the code structure and the rules S1–S4 (for example, invalid zones)

The graph must be derived from the code, with the mapping clearly shown.

Evidence Expected

A correct control flow graph with node–line mapping and an evaluation of mismatches between code and specification.

1. Node–line mapping

Consecutive statements without a decision form one node (a basic block). A decision is a node with two outgoing edges (T and F). The compound condition at line 1 (weight <= 0 or weight > 50) is split into two nodes, because the second condition is evaluated only if the first is false (short-circuit evaluation).

Table 17: Mapping of pseudocode lines to nodes of the control flow graph

Node	Line(s)	Statement	Type
A	1 (first part)	weight <= 0	Decision (T → C, F → B)
B	1 (second part)	weight > 50	Decision (T → C, F → D)
C	2	return INVALID	Exit statement
D	3	rate = 40	Statement
E	4	zone == "B"	Decision (T → F, F → G)
F	5	rate = 60	Statement
G	6	zone == "C"	Decision (T → H, F → I)
H	7	rate = 90	Statement
I	8–9	charge = 0; w = weight	Statement block
J	10	while w > 0	Loop decision (T → K, F → L)

Node	Line(s)	Statement	Type
K	11–12	charge = charge + rate; w = w − 1	Loop body (returns to J)
L	13	if express	Decision (T → M, F → P)
M	14	weight > 30	Decision (T → N, F → O)
N	15	return NOT_ALLOWED	Exit statement
O	16	charge = charge × 1.5	Statement
P	17	parcels >= 10	Decision (T → Q, F → R)
Q	18	charge = charge × 0.9	Statement
R	19	return charge	Exit statement

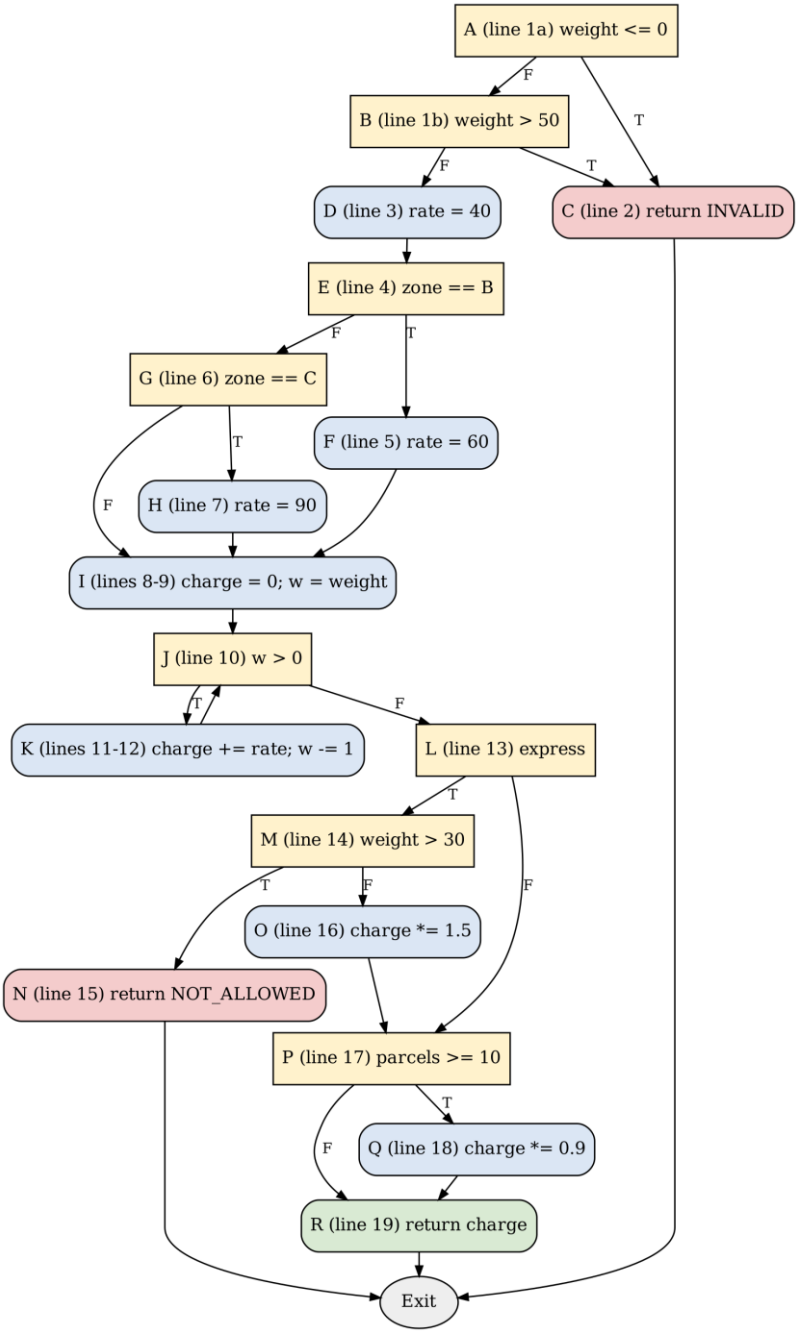


Figure 2: Control flow graph of shipCharge (decision nodes in yellow, returns in red/green)

2. Nodes and edges

The graph has 18 nodes (A–R) plus one common exit node = **19 nodes** and **26 edges**: A→B, A→C, B→C, B→D, D→E, E→F, E→G, F→I, G→H, G→I, H→I, I→J, J→K, J→L, K→J, L→M, L→P, M→N, M→O, N→Exit, O→P, P→Q, P→R, Q→R, C→Exit, R→Exit.

- **Decision nodes (8):** A, B, E, G, J, L, M, P.

- **The loop:** J (test) and K (body) form a loop; the edge K→J is the back edge. The number of iterations equals the number of started kilograms (1 to 50), so one decision node stands for up to 50 different paths.
- **The compound condition:** node A and node B together implement line 1. They add one more decision than a single line-1 node would, so the compound condition increases complexity by one (Task 8).
- **Three exits:** C (INVALID), N (NOT_ALLOWED) and R (charge); this is why a common exit node is added.

3. Code structure compared with the specification

Table 18: Mismatches between the code structure and rules S1–S4

Rule / element	Specification	Code structure and consequence
S2 invalid zone	Any zone other than A, B or C is INVALID	Nodes E and G have no "else INVALID" edge. Zone D takes the path D–E–G–I with rate 40. **A required exit is missing**; the graph shows 3 exits where the specification needs 4 outcomes (INVALID by weight, INVALID by zone, NOT_ALLOWED, charge)
S1 decimal places	Up to one decimal place	No node checks the number of decimals; the graph has no path for it
S2 started kilogram	Charge for every started kilogram (a ceiling)	Implemented by a loop (J–K) that subtracts 1 until $w \leq 0$, not by one calculation. It is correct for one-decimal weights (checked for all 500 values) but is a loop of up to 50 iterations where the specification has one formula
S3 express limit	Express allowed only up to 30 kg	Checked at M after the loop has run; the loop work for a NOT_ALLOWED parcel is wasted, but the result is correct
S4 discount	10% on the final charge for 10 or more parcels	Node P and Q after the express surcharge: matches the specification ("final charge")
Return types	Charge, INVALID or NOT_ALLOWED	The same function returns text and numbers; callers must check the type, which is a design weakness

Conclusion: the structure matches S1, S3 and S4 closely (one decision per rule) but **does not match S2**, where the specification has two behaviours (valid zone, invalid zone) and the code has only the valid ones. The graph makes the omission visible: a specification-driven test of "zone D" follows the same path as zone A.

Task 8 – Evaluate Path Testing and Cyclomatic Complexity 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Path testing aims to exercise independent paths through the code.

How many tests are needed to exercise the logic of shipCharge, and is that number enough?

Discuss:

- Cyclomatic complexity calculated by at least two methods
- A set of independent (basis) paths and test data for each path
- Infeasible paths, if any, and why
- How the loop affects path testing, and how you would test it (zero, one, many iterations)

You must show the derivation. Stating only the final number will not receive full credit.

Evidence Expected

A derived cyclomatic complexity, basis paths with test data, and an evaluation of loop testing for shipCharge.

1. Cyclomatic complexity by three methods

Table 19: Cyclomatic complexity V(G) of shipCharge

Method	Derivation	Result
Edges and nodes: V(G) = E − N + 2P	E = 26, N = 19, P = 1 connected graph: 26 − 19 + 2	**9**
Predicates: V(G) = d + 1	Decision nodes d = 8 (A, B, E, G, J, L, M, P): 8 + 1	**9**
Regions: bounded regions + 1	8 bounded regions: (A,B,C), (E,F,I,G), (G,H,I), loop (J,K), (L,M,O,P), (M,N,Exit,R,P,O), (P,Q,R), (C … Exit) + the outer region = 9	**9**

If line 1 were drawn as one decision, d = 7 and V(G) = 8; counting each condition of the compound predicate, as McCabe's method does, gives 9. V(G) = 9 means that **at least 9 tests** are needed to exercise every independent path once. The number is a lower bound for branch coverage, not a guarantee of adequacy.

2. Basis (independent) paths and test data

The baseline path is the "typical" route (BP3: valid weight, zone A, no express, parcels below 10). Each other path changes one decision outcome of the baseline.

Table 20: Nine basis paths with test data and expected results

Path	Node sequence (outcome changed from baseline)	Test data	Expected	Result
BP1	A–C–Exit (A true: weight ≤ 0)	0.0, A, No, 3	INVALID	Pass
BP2	A–B–C–Exit (B true: weight > 50)	51.0, A, No, 3	INVALID	Pass
BP3	A–B–D–E–G–I–J–K–J–L–P–R–Exit (baseline; one loop iteration)	1.0, A, No, 3	40	Pass
BP4	… E–F–I … (E true: zone B)	1.0, B, No, 3	60	Pass
BP5	… E–G–H–I … (G true: zone C)	1.0, C, No, 3	90	Pass
BP6	… L–M–O–P–R … (L true, M false: express ≤ 30)	1.0, A, Yes, 3	40 × 1.5 = 60	Pass
BP7	… L–M–N–Exit (M true: express > 30)	31.0, A, Yes, 3	NOT_ALLOWED	Pass
BP8	… L–P–Q–R … (P true: discount)	1.0, A, No, 10	40 × 0.9 = 36	Pass
BP9	… J–K–J–K–J–K–J–K–J–L … (many iterations)	3.5, A, No, 3	4 kg × 40 = 160	Pass

Nine paths are independent because each one contains an edge that no combination of the others supplies. Note that all nine pass: the basis paths find no failure in this code.

3. Infeasible paths

- **Zero loop iterations (I→J→L with J false at the first test):** infeasible. Line 1 guarantees weight > 0, so w > 0 is true the first time. In the textbook basis, "flip J" is this path. I replaced it by BP9 (many iterations), which is feasible and spans the same space (BP9 − BP3 is the loop cycle).
- **Paths that evaluate B after A is true:** do not exist, because A true goes straight to C (short-circuit).
- So 8 decisions give 9 independent paths and **all 9 can be run** after the replacement.

4. Effect of the loop and how to test it

A loop makes the number of paths depend on the data. Each started kilogram adds one iteration, so for one decimal place there are 50 possible iteration counts. The feasible complete paths are: per zone, express "No" gives 50 × 2 (parcels) = 100; express "Yes" with weight ≤ 30 gives 30 × 2 = 60; express "Yes" with weight > 30 gives 20 (always NOT_ALLOWED); so 180 per zone, 3 × 180 = 540, plus the 2 INVALID paths = **542 paths**. Testing all 542 is neither necessary nor sensible. Simple-loop testing is used instead:

Table 21: Simple-loop tests for the loop J–K

Iterations	Test weight	Expected	Remark
0 (skip the loop)	not possible	–	Infeasible: line 1 rejects weight ≤ 0
1	0.1 or 1.0 (BP3)	40	Covered by B2, B3
2	1.1	80	Covered by B4 (kilogram step)
Typical (m)	3.5 (BP9) or 12.0	160	Covered by BP9, T2
Maximum − 1	49.0 (49 iterations)	1,960	Not in the B set; 49.9 (B5) already gives 50 iterations
Maximum	50.0 (50 iterations)	2,000	B6
Maximum + 1	50.1	INVALID	B7: rejected before the loop; 51 iterations are impossible

Is 9 enough? No. Nine tests exercise every decision outcome once, but they do not test the boundaries of those decisions (BP2 uses 51 and BP7 uses 31, not 50.1 and 30.1) and they cannot find the missing zone check. In my injected-fault experiment the 9 basis tests killed only 9 of 12 faults (Task 10). They are a necessary minimum for structure, not a sufficient suite.

Task 9 – Judge Test Adequacy and Coverage 3 Marks

CO2 | Bloom's Level: K5 – Evaluate

Management wants a single coverage number to decide whether testing is “enough”.

Which coverage criterion is adequate for a module that calculates money for 40,000 parcels a day?

Discuss:

- Statement, branch, condition and multiple-condition coverage for shipCharge
- The coverage achieved by your path tests from Task 8
- The additional tests needed to reach the next stronger criterion
- Whether 100 % coverage would have found the invalid-zone defect

Your recommended criterion must be defended against cost and risk.

Evidence Expected

A justified adequacy criterion for shipCharge supported by coverage analysis of your own tests.

1. Coverage criteria applied to shipCharge

Table 22: Coverage criteria, number of items to cover and minimum tests

Criterion	Items to cover in shipCharge	Items	Smallest test set found
Statement	Lines 1–19 (all executable statements)	19	3 tests: (0.0, A, No, 3); (31.0, C, Yes, 3); (5.0, B, Yes, 10)
Branch (decision)	True and false outcome of the decisions at lines 1, 4, 6, 10, 13, 14, 17	14	4 tests: the three above + (5.0, A, No, 3)
Condition	True and false outcome of each of the 8 simple conditions (line 1 has two)	16	5 tests: the four above + (51.0, A, No, 3), because weight > 50 is evaluated only when weight ≤ 0 is false
Multiple condition	All combinations of conditions inside a decision: only line 1 is compound: TT, TF, FT, FF	3 feasible	Same 5 tests; TT (weight ≤ 0 and > 50) is impossible

The three smallest sets were found by hand and the coverage was then measured by running them on an instrumented copy of the pseudocode. The three-test statement set reaches only 11 of 14 branch outcomes and 12 of 16 condition outcomes, which shows that statement coverage is a weak criterion.

2. Coverage achieved by my test sets

Table 23: Coverage measured for each test set (instrumented execution)

Test set	Tests	Statements (19)	Branch outcomes (14)	Conditions (16)	Zone D found?
ECP (T1–T4, I1–I3)	7	19 (100%)	14 (100%)	16 (100%)	Yes (I3)
BVA (B1–B13)	13	17 (89.5%)	12 (85.7%)	14 (87.5%)	No
Decision table (DT1–DT8)	8	18 (94.7%)	13 (92.9%)	14 (87.5%)	No
Basis paths (BP1–BP9, Task 8)	**9**	**19 (100%)**	**14 (100%)**	**16 (100%)**	**No**
Final suite (Task 11)	16	19 (100%)	14 (100%)	16 (100%)	Yes (F16)

The nine path tests of Task 8 give 100% statement, branch, condition and multiple-condition coverage (the feasible combinations of line 1 are covered by BP1, BP2 and BP3). BVA and the decision table leave lines 5 and 7 (zones B and C) unexecuted because they use mostly zone A; this is the weakness of boundary-only testing.

3. Next stronger criterion

The strongest practical criterion above multiple-condition coverage is **path coverage**, which here means the 542 paths of Task 8. The basis paths cover 9 of 542 (1.7%). A practical step towards it is basis paths plus simple-loop testing (the loop is the source of the path explosion). It needs **four more tests**: (1.1, A, No, 3) for two iterations, (50.0, C, No, 10) for 50 iterations, (30.0, A, Yes, 3) for 30 iterations and (30.1, A, Yes, 3) for 31 iterations followed by NOT_ALLOWED. These four are already in my boundary set (B4, B6, B9, B10), so 13 tests reach this level.

4. Would 100% coverage have found the invalid-zone defect?

No. The defect is a missing branch. The nine path tests already reach 100% of statements, branches and conditions, and none of them uses an invalid zone. A zone "D" test follows the existing path D–E–G–I, which is already "covered" by zone A: coverage cannot reveal that a decision is absent. Even 100% path coverage would not guarantee detection, because a zone D test is needed together with an expected result from the specification. Coverage is therefore a measure of how much code was exercised, not of whether the code is correct and complete.

5. Recommended criterion

For a module that calculates money for 40,000 parcels a day I recommend **100% branch coverage combined with 100% condition coverage** (which, because only line 1 is compound, equals multiple-condition coverage here), together with the specification-based tests of Tasks 3–5. This costs 5 tests at minimum and is reached by the existing sets at no extra cost. Statement coverage is too weak (11 of 14 branches in my 3-test set); full path coverage (542 tests) is too expensive and, as shown, still misses omissions. The coverage figure should be reported with the black-box evidence, never as the only “enough” number.

Task 10 – Compare Black-Box and White-Box Effectiveness 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

You now have black-box tests (Tasks 3–6) and white-box tests (Tasks 7–9).

Which approach actually found more of the important defects in shipCharge?

Discuss:

- The defects found by black-box tests only, white-box tests only, and both
- Reasons why each approach missed what it missed
- Overlapping tests that can be removed without losing defect-finding power
- Whether this result would hold for other modules

You are expected to interpret the comparison, not just count tests.

Evidence Expected

A reasoned comparison of black-box and white-box effectiveness for shipCharge, supported by your own test results.

1. How the comparison was made

The pseudocode contains **one real defect**: an invalid zone is not rejected (D1). One defect is too few for a comparison, so I also injected 12 non-equivalent faults (mutants) into a copy of the pseudocode, one at a time, and ran each test set against each mutant. A mutant is "found" when at least one test gives a different result from the correct code. A 13th mutant (applying the discount before the surcharge) gives identical results for every input because multiplication is commutative; it is an equivalent mutant and is excluded.

Table 24: Injected faults (mutants) of shipCharge

ID	Change in the code	Likely real-world cause
M1	line 1: weight <= 0 → weight < 0	Wrong boundary at the lower limit
M2	line 1: weight > 50 → weight >= 50	Wrong boundary at the upper limit
M3	zone B rate 60 → 50	Wrong rate
M4	zone C rate 90 → 80	Wrong rate
M5	loop: w > 0 → w >= 1	Started kilogram rounded down
M6	line 14: weight > 30 → weight >= 30	Wrong express limit
M7	line 14: weight > 30 → weight > 31	Wrong express limit
M8	surcharge × 1.5 → × 1.4	Wrong surcharge

ID	Change in the code	Likely real-world cause
M9	parcels >= 10 → parcels > 10	Wrong discount threshold
M10	discount × 0.9 → × 0.95	Wrong discount
M13	loop step w − 1 → w − 0.9	Loop counter error
M14	weight > 30 test applied even without express	Misplaced condition

2. Results

Table 25: Defects found by each test set

Test set	Tests	Real defect D1	Injected faults found (of 12)	Faults missed
ECP	7	Yes	8	M1, M2, M6, M7
BVA	13	No	10	M3, M4
Decision table	8	No	7	M1, M2, M5, M6, M7
Black-box total (ECP + BVA + DT)	**28**	**Yes**	**12**	**none**
White-box (basis paths BP1–BP9)	**9**	**No**	**9**	**M2, M6, M14**
Final suite (Task 11)	16	Yes	12	none

Table 26: Defects found by black-box only, white-box only, and both

Found by	Defects	Explanation
Black-box only	D1 (zone D accepted), M2 (weight ≥ 50), M6 (weight ≥ 30), M14 (limit applied without express)	BVA tests 50.0 and 30.0, ECP has the invalid zone, and tests of heavy parcels without express (T3, B11) exist only in the black-box sets
White-box only	**None**	Every fault that the nine path tests found, the black-box set found too
Both	M1, M3, M4, M5, M7, M8, M9, M10, M13	Each is a plain wrong value or wrong decision on a path that is executed by tests of both kinds

3. Why each approach missed what it missed

- **White-box tests missed D1:** the defect is an omission. There is no code for the missing check, hence nothing to cover. The test data of the basis paths were chosen to flip decisions, not to represent invalid zones.
- **White-box tests missed M2, M6 and M14:** a path test needs a value that drives a decision, not the value at the edge: BP2 uses 51.0 and BP7 uses 31.0, which behave the same under > and ≥. M14 is missed because the basis tests use heavy parcels only together with express.
- **Black-box tests missed nothing important individually in the full set,** but each technique alone missed something: ECP missed the four boundary faults (representatives are mid-class), BVA missed the two zone rates (all boundary tests used zone A), and the decision table missed faults outside the S3/S4 interaction.
- **White-box value, however:** it showed that the black-box set leaves no code unexecuted, found that zero iterations are infeasible, and noticed the wasted loop before NOT_ALLOWED and the loop-with-floating-point risk. These are code observations rather than failures.

4. Overlap that can be removed

The union of the black-box and white-box tests has 28 + 9 = 37 tests. After comparing classes, rules, paths and expected results, 21 can be removed without losing any detected fault, any ECP class, any decision rule, any basis path or any coverage, leaving the 16-test final suite of Task 11:

- All nine basis tests duplicate black-box tests with the same path and class (BP1 = B1, BP2 = B7, BP3 = B3, BP4 = F7, BP5 = F6, BP6 = B8/B9, BP7 = B10, BP8 = B13, BP9 = B5/B6).
- ECP tests T1, T3 and T4 are covered by F12, F14 and F9; I1 and I2 are covered by the boundary tests 0.0 and 50.1 of the same classes.
- B8 (29.9 express) is identical in class and expected charge to B9 (30.0); DT1–DT4 duplicate F12–F14 and F10.

5. Would the result hold for other modules?

Not necessarily. In shipCharge the code is short and follows the specification rule by rule, so the specification-based tests exercise all of the code and white-box testing adds measurement but no new defects. In modules with logic that is **not** in the specification (for example complex

rounding, caching, error handling, or concurrent updates) black-box tests do not know the hidden paths, and white-box tests find defects that black-box tests cannot. The tracking module is different again: its defects are about sequences of events, which neither input-based black-box nor path testing handles well, so state-transition tests are needed. The conclusion for shipCharge is that black-box found more of the important defects, but the right lesson is that the techniques complement each other.

Task 11 – Recommend and Defend the Final Test Suite 4 Marks

CO2 | Bloom's Level: K5 – Evaluate

Based on your analysis, recommend the final regression test suite for SwiftShip.

Discuss:

- The test cases you include and the technique each comes from
- Why this suite is better than the alternatives
- The coverage and the rule combinations it achieves
- The number of test cases and the effort needed to run them
- Known gaps and risks that remain

Your recommendation must be based on evidence from your own test design. The largest suite is not automatically the best.

Evidence Expected

A technically justified final test suite supported by black-box, white-box and coverage evidence.

1. The recommended suite

Table 27: Final regression suite for shipCharge (16 tests)

ID	Input (weight, zone, express, parcels)	Technique / purpose	Rule	Expected result
F1	0.0, A, No, 3	BVA min−; ECP W3; BP1	–	INVALID
F2	50.1, A, No, 3	BVA max+; ECP W4; BP2	–	INVALID
F3	0.1, A, No, 3	BVA min; loop 1; BP3	R1	40
F4	1.0, A, No, 3	BVA kilogram step	R1	40
F5	1.1, A, No, 3	BVA kilogram step; loop 2	R1	80
F6	50.0, C, No, 3	BVA max; zone C; BP5; loop 50	R3	4,500
F7	49.9, B, No, 3	BVA max−; zone B; BP4	R3	3,000
F8	30.0, A, Yes, 3	BVA express limit; BP6	R5	1,800
F9	30.1, A, Yes, 3	BVA express limit+; BP7	R7	NOT_ALLOWED

ID	Input (weight, zone, express, parcels)	Technique / purpose	Rule	Expected result
F10	30.1, B, No, 3	BVA; heavy without express (kills M14)	R3	1,860
F11	12.0, B, Yes, 10	ECP; decision table	R6	972
F12	5.0, A, No, 9	BVA parcels 9	R1	200
F13	5.0, A, No, 10	BVA parcels 10; BP8	R2	180
F14	40.0, B, No, 10	ECP W2; decision table	R4	2,160
F15	40.0, C, Yes, 10	Decision table (missed by ECP+BVA)	R8	NOT_ALLOWED
F16	5.0, D, No, 3	ECP invalid zone Z4	–	INVALID (currently **fails**)

The tracking module adds the 14 tests of Task 6 (S1–S3 and IT1–IT11), so the **whole regression suite has 30 tests**. F16 fails on the current code and must stay in the suite until the defect is fixed; after the fix it protects against its return.

2. Why this suite is better than the alternatives

Table 28: Comparison with alternative suites for shipCharge

Alternative	Tests	Injected faults found (of 12)	Real defect D1 / coverage	Verdict
ECP only	7	8	Yes / 100%	Misses all four boundary faults
BVA only	13	10	No / 87%	Misses zone rates and D1
Basis paths only	9	9	No / 100%	Misses D1, M2, M6, M14
All black-box tests (ECP + BVA + DT)	28	12	Yes / 100%	Complete but 12 duplicates
All 542 paths	542	12	Yes / 100%	Not repeatable before every release
1,000 random tests	1,000	Not measured	Possibly / not measured	Needs an oracle; boundaries hit only by chance (Task 2)
Recommended suite	**16**	**12**	**Yes / 100%**	**Same defect-finding as 28 tests**

3. Coverage and rule combinations achieved

- **ECP:** all 9 valid classes, the NOT_ALLOWED behaviour and the 3 invalid classes with a stated result.
- **BVA:** every boundary of S1–S4 (0, 0.1, 30, 30.1, 49.9, 50, 50.1, the kilogram steps, parcels 9 and 10).
- **Decision table:** all 8 rules R1–R8.
- **White-box:** all 9 basis paths; 100% statements (19/19), branches (14/14) and conditions (16/16).
- **Fault detection:** 12 of 12 injected faults and the real defect D1.

4. Number of tests and effort

The suite has 30 tests (16 for shipCharge + 14 for tracking). If run by hand at about 3 minutes per test (my assumption, including data entry and checking), it takes about 90 minutes; automated with a data table of inputs and expected results, each test takes about a second or two (also an assumption), so the whole suite runs in about a minute. I therefore recommend automating it fully. It is 57% smaller than the 28 + 9 = 37 tests it was derived from (37 tests become 16 for shipCharge) and finds the same faults.

5. Known gaps and risks

- **Specification gaps:** more than one decimal place (W5), negative or fractional parcels (P3), lower-case zone letters, rounding of the final charge to 2 decimals and the currency unit are not defined; the expected values assume exact arithmetic. For floating point I would compare charges with a tolerance of ±0.005.
- **Only 11 of 48 invalid transitions** are tested; the other 37 are low risk but untested.
- **Evidence limit:** the injected faults are my own selection; the suite is proven only against them and the real defect, not against all possible faults. A second real defect would exist only if another omission is in the specification.
- **Not covered:** performance for 40,000 parcels a day, concurrent bookings, security and integration with payment or tracking events.

DAY-4 DESIGN CHALLENGE

Task 12 – Re-evaluate and Adapt the Test Suite 8 Marks

CO2 | Bloom's Level: K5 – Evaluate

Changed Requirement

SwiftShip will add a fuel surcharge of 8 % on Zone C parcels and a weekend delivery option that adds 50 per parcel.

The developers have changed shipCharge by adding two new decisions after line 16.

The suite must still run within 30 minutes before every release.

Re-evaluate your original solution in response to these changed conditions. You may: RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Discuss:

- What has changed in the specification and the code
- Which equivalence classes, boundaries and decision-table rules are affected
- How the control flow graph and cyclomatic complexity change
- Why the 30-minute limit matters

Evidence Expected

A clear analysis of how the changes affect the original test design.

What has changed in the specification and the code

I assume the two new decisions are placed after line 16 and before line 17, in the order of the requirement: first the fuel surcharge, then the weekend charge, then the discount. The function receives a new input, weekend (true/false).

Table 29: Original and changed shipCharge

Item	Original	Changed
Specification	S1–S4	S1–S4 + S5: zone C charge × 1.08; S6: weekend delivery adds 50 per parcel
Inputs	weight, zone, express, parcels	weight, zone, express, parcels, **weekend**

Item	Original	Changed
Code after line 16	if parcels >= 10 → × 0.9 (line 17)	New line 16a: if zone == "C" then charge × 1.08; new line 16b: if weekend then charge + 50; then line 17 discount
Order of operations	rate × kg → × 1.5 → × 0.9	rate × kg → × 1.5 → × 1.08 (zone C) → + 50 (weekend) → × 0.9
Questions for the specification	–	Does the fuel surcharge apply to the base or after express? Is the 50 discounted by S4? Is weekend allowed with express? I assume the order above (the discount applies to the "final charge", so 50 is discounted too) and confirm it with the business

Equivalence classes, boundaries and decision-table rules affected

- **Equivalence classes:** a new input class pair weekend = Yes / No; the zone C class now has its own behaviour (× 1.08), so Z3 must be tested with both express and discount values. Invalid classes (W3, W4, Z4) are unchanged.
- **Boundaries:** no new numeric boundary; weight boundaries 0, 30 and 50 and parcels 9/10 are unchanged. But every expected charge on zone C at a boundary changes (for example 50.0 kg zone C: 4,500 → 4,860).
- **Decision table:** conditions grow from 3 to 5 (add zone = C and weekend). The 8 rules become up to 25 once the NOT_ALLOWED rules are merged (24 charge rules + NOT_ALLOWED). Rules R1–R6 all now have a zone-C and a weekend variant; R7 and R8 (NOT_ALLOWED) return before the new decisions and are unchanged.

Control flow graph and cyclomatic complexity

Table 30: Effect of the change on the control flow graph

Measure	Before	After	Reason
Nodes (with exit)	19	23	2 new decisions + 2 new statements
Edges	26	32	7 new edges, 1 old edge removed (O→P)
Decision nodes	8	10	+ zone == "C" (16a), + weekend (16b)

Measure	Before	After	Reason
V(G) = E − N + 2	9	**11**	32 − 23 + 2 = 11; predicates: 10 + 1 = 11
Feasible complete paths	542	1,022	each non-NOT_ALLOWED path doubles with weekend: 3 × (50×4 + 30×4 + 20) + 2

There is an important twist: the new decision "zone == C" depends on the old decision at line 6 ("zone == C"). A path with line 6 true and fuel false (or the reverse) is **infeasible**. So of the 11 theoretical basis paths only **10 are feasible**; V(G) over-estimates the independent tests. This is a design smell: the zone should be examined once (set the rate and the fuel flag together).

Why the 30-minute limit matters

The suite runs before every release, so it must fit the time that the release process allows. At about 3 minutes per manual test, even the 34 tests of the modified suite would take about 102 minutes, more than three times the limit. Only an automated suite fits (about a minute); with the budget of 30 minutes (1,800 s) and 2 s per test, the ceiling is about 900 tests. The limit therefore forbids manual regression, path coverage of all 1,022 paths, and unlimited growth of the suite whenever a rule is added.

B. Evaluation of the Original Suite **2 Marks**

CO2 | Bloom's Level: K5 – Evaluate

Discuss:

- Which existing tests are still valid, need new expected results, or are obsolete
- Which new paths and rule combinations are not covered
- Whether the coverage criterion can still be met

Evidence Expected

A technically supported evaluation of the original suite under the new conditions.

Status of the existing tests

Table 31: Status of the 30 existing tests after the change

Group	Tests	Status
shipCharge tests with unchanged result (F1–F5, F7–F14, F16)	14	**Still valid** once the new argument weekend = No is added; expected results are unchanged because zone C is not involved or the result is NOT_ALLOWED / INVALID
F6 (50.0 kg, zone C, no express, 3 parcels)	1	**New expected result:** 4,500 × 1.08 = 4,860
F15 (40.0 kg, zone C, express, 10 parcels)	1	Still valid (NOT_ALLOWED); I change it to weekend = Yes to prove that no weekend charge is added to a rejected request
Tracking tests S1–S3, IT1–IT11	14	**Still valid**; the tracking module is not changed
Obsolete tests	0	None: no rule was removed

What the old suite does not cover

Table 32: Coverage of the changed code by the old suite and the modified suite

Measure	Old 16 tests	Modified 20 tests	Remark
Statements	20 of 21 (95.2%)	21 of 21 (100%)	Line 16b "charge + 50" never executed by the old suite
Branch outcomes	17 of 18 (94.4%)	18 of 18 (100%)	Weekend = true outcome missing
Injected faults found (15: 12 old + 3 new)	13	15	New faults: fuel on zone B (N1), weekend added after the discount (N2), weekend charge 5 instead of 50 (N4)
Rule combinations (pairs of 4 factors: zone C, express, weekend, discount)	Zone C × weekend, express × weekend, discount × weekend not covered	All 24 pairs covered	Pairwise coverage

The old suite finds the fuel-on-zone-B fault (F7 uses zone B) but **cannot** find an incorrect weekend charge: the combinations (weekend, discount) and (zone C, weekend) were never run. The two surviving faults (N2: 50 added after the discount, and N4: 5 instead of 50) are exactly the kind of money errors the suite must catch.

Can the coverage criterion still be met?

Yes. The criterion chosen in Task 9, 100% branch and condition coverage, needs only **one more test** (weekend = Yes) to be met again, because the zone C true and false outcomes are already reached by F6 and the other tests. The stronger need is the combination coverage, which requires four new tests (Task 12C).

C. Adaptation or Redesign — 2 Marks

CO2 | Bloom's Level: K5 – Evaluate

Decide whether to retain, modify or redesign the test suite.

If you modify or redesign it:

- Describe the new and changed test cases
- Explain which techniques you used to derive them
- Show the effect on coverage and the number of test cases

If you retain the original suite, provide sufficient justification for why no change is necessary.

Evidence Expected

An appropriate test-design response to the changed requirement with clear reasoning.

Decision: MODIFY the suite

I do **not retain** the suite, because one expected result is wrong (F6), line 16b and one branch are not covered (95.2% statements, 94.4% branches) and two realistic faults survive. I do **not redesign** it, because 29 of the 30 tests still stand, the techniques (ECP, BVA, decision table, basis paths, state transition) still apply, and the change touches only one function in one place. The change is small: one modified expected result, one modified input and four new tests.

New and changed test cases

Table 33: Changed and new tests (weekend is the fifth input)

Test	Input (weight, zone, express, parcels, weekend)	Technique	Expected result and calculation	Status
F6	50.0, C, No, 3, No	Changed result	50 × 90 = 4,500; × 1.08 = **4,860**	Changed
F15	40.0, C, Yes, 10, **Yes**	Decision table	**NOT_ALLOWED** (no fuel, no weekend charge)	Changed input
W1	10.0, A, No, 3, Yes	ECP: weekend class	10 × 40 = 400; + 50 = **450**	New

Test	Input (weight, zone, express, parcels, weekend)	Technique	Expected result and calculation	Status
W2	10.0, B, Yes, 10, Yes	Decision table: weekend × discount	10 × 60 = 600; × 1.5 = 900; + 50 = 950; × 0.9 = **855**	New
W3	10.0, C, Yes, 3, No	Decision table: fuel × express	900 × 1.5 = 1,350; × 1.08 = **1,458**	New
W4	10.0, C, Yes, 10, Yes	All four changes together; basis path with both new decisions	1,350 × 1.08 = 1,458; + 50 = 1,508; × 0.9 = **1,357.20**	New

Because of floating-point arithmetic, results such as 919.8000000000001 can occur, so the comparison must use a tolerance of ±0.005 (or integer paise).

Techniques used to derive them

- **Decision table (extended):** factors zone C, express, weekend and discount; instead of all 24 charge rules I chose tests that cover **every pair of factors** (24 pairs): (C, express) by F6, W3, F8, F3; (C, weekend) by W4, F6, W1, F3; (C, discount) by W4, F6, F13, F3; (express, weekend) by W2, F8, W1, F3; (express, discount) by F11, F8, F13, F3; (weekend, discount) by W2, W1, F13, F3.
- **ECP:** the new weekend input has two classes, covered by W1 and F3 (weekend = No).
- **Basis path testing:** V(G) = 11 gives two new independent paths: weekend true (W1) and fuel true, which is feasible only together with zone C (F6, W4). The other 9 basis paths remain covered.
- **BVA:** unchanged; no new numeric boundary. The existing boundary tests are re-run with weekend = No.

Effect on coverage and size

Table 34: Effect of the modification

Measure	Original	Modified	Change
shipCharge tests	16	20	+4
Tracking tests	14	14	0
Total tests	30	34	+4 (+13%)
Statement coverage	95.2% (after change)	100% (21/21)	+4.8 points

Measure	Original	Modified	Change
Branch coverage	94.4% (after change)	100% (18/18)	+5.6 points
Injected faults found	13 of 15	15 of 15	+2
Decision rules (pairs) covered	Not covered for weekend and fuel	All pairs covered	–

An optional extra: because the inputs are few, a data-driven nightly table of 6 representative weights × 4 zones × express × parcels class × weekend = 192 combinations would take about 192 × 2 s = 6.4 minutes, which still fits in 30 minutes, **provided** the expected values come from an independent calculator and not from shipCharge itself (Task 2). I recommend it as a supplement, not as a replacement for the 34-test regression suite that people can review.

D. Evidence-Based Defence **2 Marks**

CO2 | Bloom's Level: K5 – Evaluate

Defend your final test suite using evidence. Use:

- Equivalence classes and boundaries
- Decision-table combinations
- Cyclomatic complexity and basis paths
- Coverage achieved
- Execution time limit

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final test suite integrating test design evidence and the changed constraints.

Defending the modified suite

Table 35: Evidence supporting the decision to modify the suite

Evidence	What it shows	Effect on my decision
Equivalence classes and boundaries	The 12 boundary and class decisions of S1–S4 are unchanged; only zone C values change (F6: 4,500 → 4,860); the ECP + BVA suite found 12 of 12 injected faults and D1	Keep all of ECP and BVA; change one expected result

Evidence	What it shows	Effect on my decision
Decision-table combinations	The old table covers S3 and S4 (8 rules, all covered by the suite). The change adds 2 factors, and 3 pairs of factors are untested (weekend with discount, zone C, express)	Add W1–W4 using pairwise coverage; the full table of 24 rules is not needed
Cyclomatic complexity and paths	V(G) rose from 9 to 11 (10 feasible because the new zone test duplicates line 6); paths grew from 542 to 1,022	Two new basis paths are covered by W1 and F6/W4; all-path testing is rejected
Coverage achieved	Old suite on new code: 95.2% statements, 94.4% branches, 13 of 15 faults; modified suite: 100%, 100%, 15 of 15	The criterion (100% branch + condition) is met again with 4 more tests
Execution time limit (30 min)	34 tests take about a minute automated (about 102 min by hand); the ceiling is about 900 tests	Fully automate; suite stays far below the limit; no need to drop tests

Reasoning. Retaining the suite would leave a wrong expected result (F6) and untested money logic: a wrong weekend charge or an undiscounted 50 would reach customers on every weekend booking of 40,000 parcels a day. A redesign would discard 29 tests that the evidence still supports and cost extra effort for no gain. The modification adds only four tests, restores 100% branch coverage and finds all 15 injected faults within a minute of run time.

Risks I accept. The order of the surcharges and the discount on the 50 are assumptions that must be confirmed with the business (a different order changes W2 and W4 expected values). The weekend input is assumed to be a simple yes/no. The faults used for the evidence are my own, and the suite is only as good as those faults and the real defect D1; it is a minimum regression suite, not proof of correctness. The real invalid-zone defect (F16) still fails and must be fixed before the release.